

```
In [1]: # 18/12/2025 Running TF-COMBS to find what TFs co-occur with STAT3 on the top 300 differentially accessible peaks between WT and SteE KO Salmc

# Using new tables Peter sent me for iBMDMs
#%pip install --upgrade pip
#%pip install git+https://github.com/loosolab/TF-COMB

# NB: need to comment out this line from turtle import end_fill from the utils.py file
# NB: also need to replace var_name=['TF2'] with var_name='TF2' in objects.py so market_basket works

# needed for os.path.join
import os

# use to show all outputs from a cell, not just the last one
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

import pandas as pd

from tfcomb import CombObj, DiffCombObj

import tfcomb
print("tfcomb version: ", tfcomb.__version__)

tfcomb version: 1.1.1
```

```
In [2]: # set up a CombObj
background = CombObj()

# search for motifs
dir_path = "/scratch/prj/id_hill_sims_wellcda/SteE_revisions/"
background.TFBS_from_motifs(regions = os.path.join(dir_path, "TF-COMBS/ATACSeq.OCRs.DESeq2.ssaV_vs_WT.all_regions.bed"),
    motifs = os.path.join(dir_path, "TF-COMBS/HOCOMOCov11_core_MOUSE_mono_meme_format.meme"),
    genome = os.path.join(dir_path, "genome.fa"),
    #motif_pvalue = 0.01, # less strict than default 0.00001
    threads = 10)

# overwrite counts with larger window
background.count_within(max_dist=500,
    #binarize = True, # if True then each cooccurrence is only counted once per window
    threads=10)

background.market_basket(threads=10)

INFO: Scanning for TFBS with 10 thread(s)...
INFO: Progress: 10%
INFO: Progress: 20%
INFO: Progress: 30%
INFO: Progress: 40%
INFO: Progress: 50%
INFO: Progress: 60%
INFO: Progress: 70%
INFO: Progress: 80%
INFO: Progress: 90%
INFO: Finished!
INFO: Processing scanned TFBS
INFO: Identified 1003908 TFBS (356 unique names) within given regions
INFO: Setting up binding sites for counting
INFO: Counting co-occurrences within sites
INFO: Counting co-occurrence within background
INFO: Progress: 10%
INFO: Progress: 20%
INFO: Progress: 30%
INFO: Progress: 40%
INFO: Progress: 50%
INFO: Progress: 60%
INFO: Progress: 70%
INFO: Progress: 80%
INFO: Progress: 90%
INFO: Finished!
INFO: Done finding co-occurrences! Run .market_basket() to estimate significant pairs
INFO: Market basket analysis is done! Results are found in <CombObj>.rules

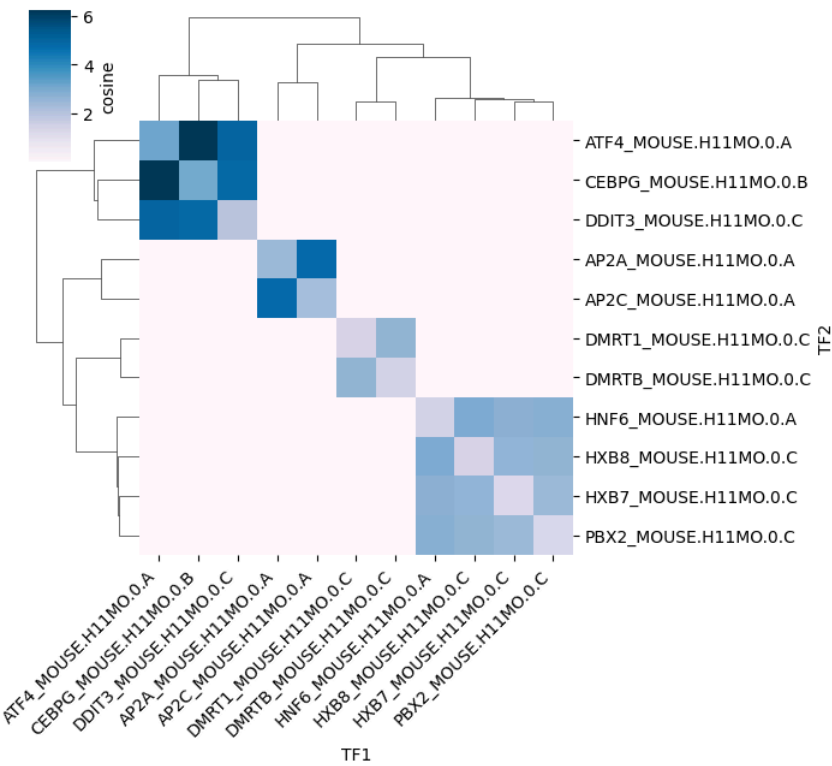
In [3]: background.rules
```

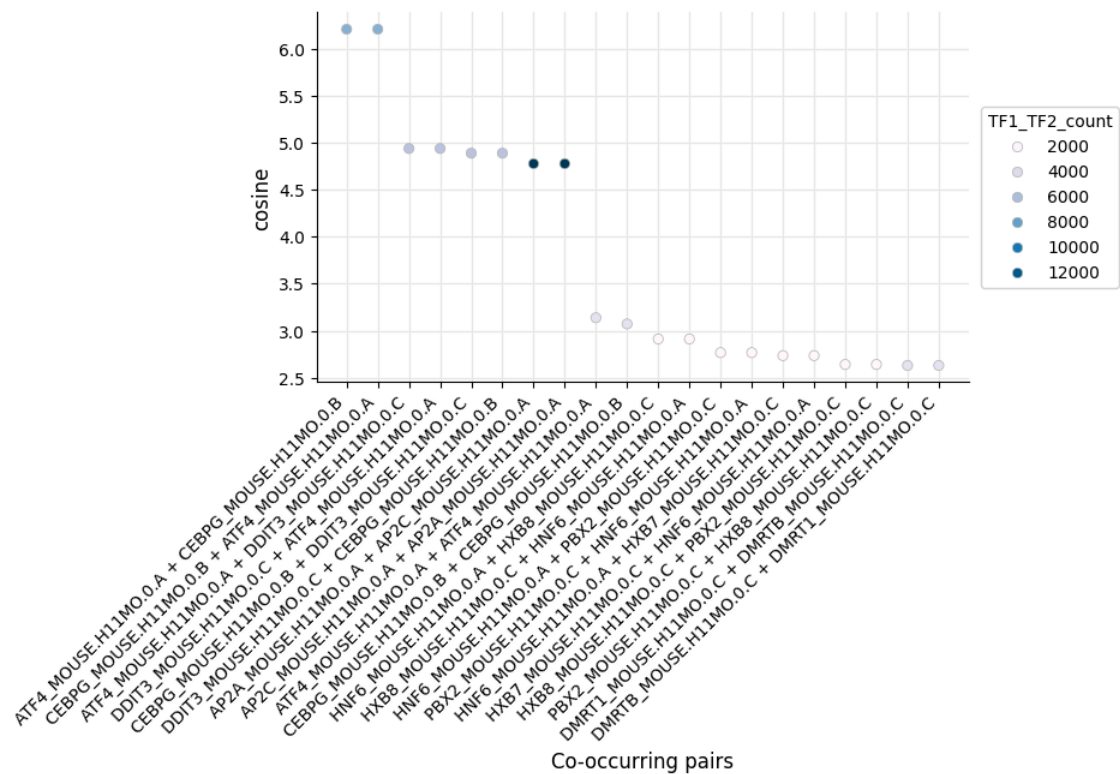
Out [3]:

	TF1	TF2	TF1_TF2_count	TF1_count	TF2_count	cosine	zscore
ATF4_MOUSE.H11MO.0.A- CEBPG_MOUSE.H11MO.0.B	ATF4_MOUSE.H11MO.0.A	CEBPG_MOUSE.H11MO.0.B	6993	1112	1141	6.208237	1125.945121
CEBPG_MOUSE.H11MO.0.B- ATF4_MOUSE.H11MO.0.A	CEBPG_MOUSE.H11MO.0.B	ATF4_MOUSE.H11MO.0.A	6993	1141	1112	6.208237	1125.945121
ATF4_MOUSE.H11MO.0.A- DDIT3_MOUSE.H11MO.0.C	ATF4_MOUSE.H11MO.0.A	DDIT3_MOUSE.H11MO.0.C	5447	1112	1094	4.938514	889.239659
DDIT3_MOUSE.H11MO.0.C- ATF4_MOUSE.H11MO.0.A	DDIT3_MOUSE.H11MO.0.C	ATF4_MOUSE.H11MO.0.A	5447	1094	1112	4.938514	889.239659
CEBPG_MOUSE.H11MO.0.B- DDIT3_MOUSE.H11MO.0.C	CEBPG_MOUSE.H11MO.0.B	DDIT3_MOUSE.H11MO.0.C	5462	1141	1094	4.888777	791.866088
...	...	...	...	...	...	...	...
MAX_MOUSE.H11MO.0.A- HXB8_MOUSE.H11MO.0.C	MAX_MOUSE.H11MO.0.A	HXB8_MOUSE.H11MO.0.C	1	781	653	0.001400	-3.446028
HNF6_MOUSE.H11MO.0.A- MYC_MOUSE.H11MO.0.A	HNF6_MOUSE.H11MO.0.A	MYC_MOUSE.H11MO.0.A	1	593	891	0.001376	-4.574763
MYC_MOUSE.H11MO.0.A- HNF6_MOUSE.H11MO.0.A	MYC_MOUSE.H11MO.0.A	HNF6_MOUSE.H11MO.0.A	1	891	593	0.001376	-4.574763
ATF1_MOUSE.H11MO.0.B- HLF_MOUSE.H11MO.0.C	ATF1_MOUSE.H11MO.0.B	HLF_MOUSE.H11MO.0.C	1	831	745	0.001271	-3.793753
HLF_MOUSE.H11MO.0.C- ATF1_MOUSE.H11MO.0.B	HLF_MOUSE.H11MO.0.C	ATF1_MOUSE.H11MO.0.B	1	745	831	0.001271	-3.793753

126471 rows x 7 columns

```
In [4]: _ = background.plot_heatmap()  
_ = background.plot_bubble()
```





```
In [5]: # filter for STAT3 containing pairs
background.TFBS[0].__dict__

# look at all TFBS sites containing STAT3
# this way returns a list of TFBS object
stat3_tfbs = [x for x in background.TFBS if "STAT3" in x.name]
stat3_tfbs[:5]

len(stat3_tfbs)
## 1,773 stat3 sites

# get all STAT3 motif variants
stat3_variants = sorted(set(x.name for x in background.TFBS if "STAT3" in x.name))
print(stat3_variants)

# filter the significant co-occurrence pairs to those with STAT3 co-occurrence
sorted(set(x.name for x in background.TFBS if "STAT3" in x.name))

stat3_pairs = background.select_TF_rules(["STAT3_MOUSE.H11MO.0.A"])
stat3_pairs.rules.head()

Out[5]: {'chrom': 'chr1',
'start': 4256793,
'end': 4256837,
'name': 'ZN322_MOUSE.H11MO.0.B',
'score': 11.80539,
'strand': '+',
'get_width': <function tobias.utils.regions.OneRegion.get_width(self)>}}

Out[5]: [chr1 6444852 6444864 STAT3_MOUSE.H11MO.0.A 10.57249 +,
chr1 6624761 6624773 STAT3_MOUSE.H11MO.0.A 10.66145 -,
chr1 6691520 6691532 STAT3_MOUSE.H11MO.0.A 9.33525 -,
chr1 13206891 13206903 STAT3_MOUSE.H11MO.0.A 9.37977 +,
chr1 13374400 13374412 STAT3_MOUSE.H11MO.0.A 9.65438 -]

Out[5]: 1773
['STAT3_MOUSE.H11MO.0.A']

Out[5]: ['STAT3_MOUSE.H11MO.0.A']
INFO: Selected 1 rules
INFO: Creating subset of object

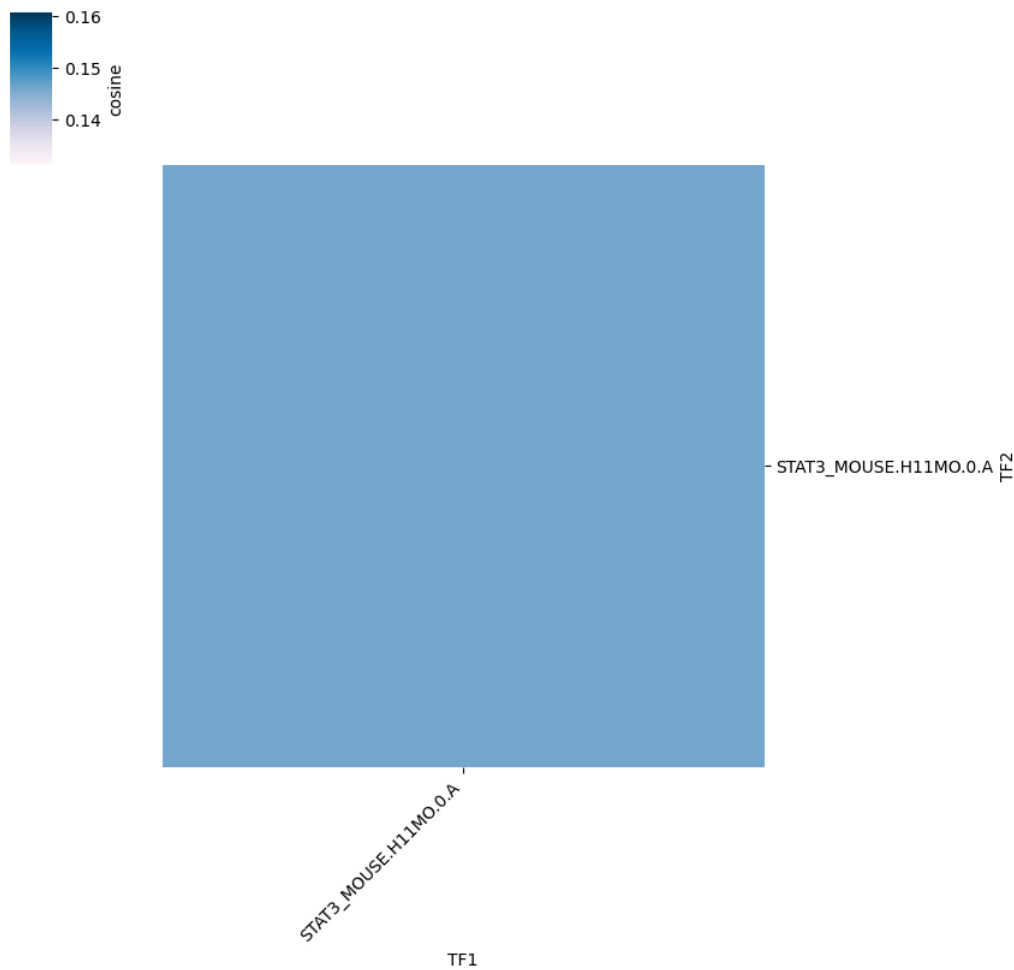
Out[5]:
```

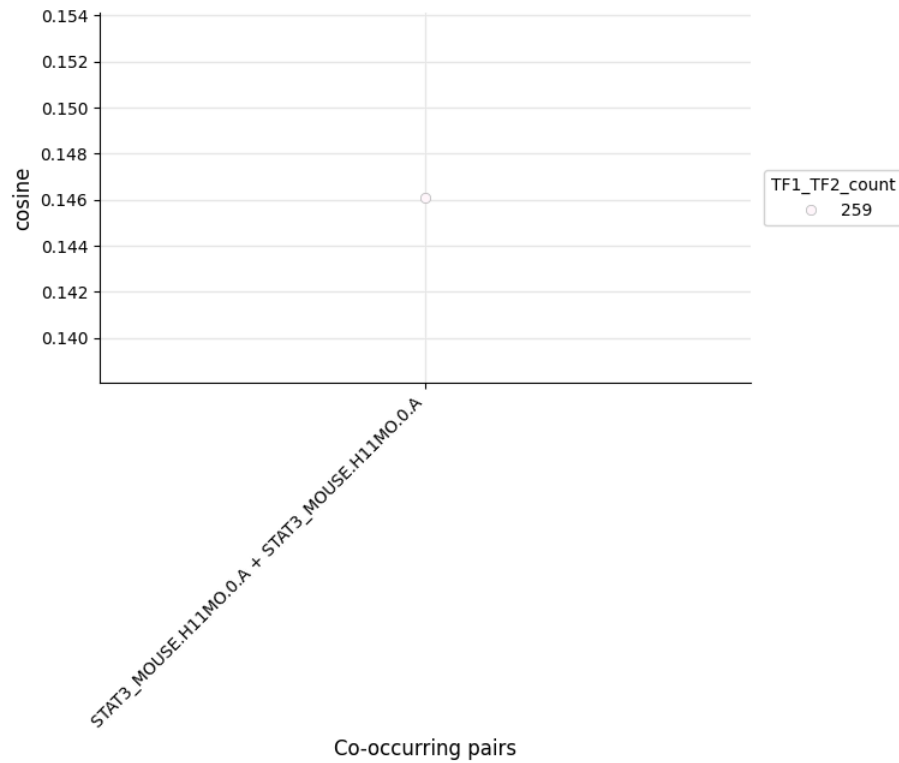
	TF1	TF2	TF1_TF2_count	TF1_count	TF2_count	cosine	zscore
	STAT3_MOUSE.H11MO.0.A- STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	259	1773	1773	0.14608 28.039597

```
In [6]: background.__dict__.keys()
```

```
Out[6]: dict_keys(['verbosity', 'logger', 'prefix', 'TFBS', 'TF_names', 'count_names', 'TF_counts', 'pair_counts', 'bg_counts_mean', 'bg_counts_std',  
'rules', 'network', 'min_dist', 'max_dist', 'directional', 'min_overlap', 'max_overlap', 'anchor', 'anchor_to_int', 'distObj', 'n_TFBS', 'stranded', 'binarize', 'name_to_idx', '_sites', 'zscore', 'TF_table'])
```

```
In [7]: _ = stat3_pairs.plot_heatmap()  
_ = stat3_pairs.plot_bubble()
```





Co-occurring pairs

```
In [8]: # do it for WT vs SPI2 opened iBMDM peaks
# set up a CombObj
opened = CombObj()

# search for motifs
dir_path = "/scratch/prj/id_hill_sims_wellcda/SteE_revisions/"
opened.TFBS_from_motifs(regions = os.path.join(dir_path, "TF-COMBS/ATACSeq.OCRs.DESeq2.ssaV_vs_WT.SPI2_opened_regions.bed"),
                        motifs = os.path.join(dir_path, "TF-COMBS/HOCOMOCov11_core_MOUSE_mono_meme_format.meme"),
                        genome = os.path.join(dir_path, "genome.fa"),
                        #motif_pvalue = 0.01, # default is 0.00001
                        threads = 4)

# overwrite counts with larger window
opened.count_within(max_dist=500,
                   #binarize = True, # if True, then each cooccurrence is only counted once per window
                   threads=10)

opened.market_basket(threads=10)

opened.rules
_ = opened.plot_heatmap()
_ = opened.plot_bubble()
```

```

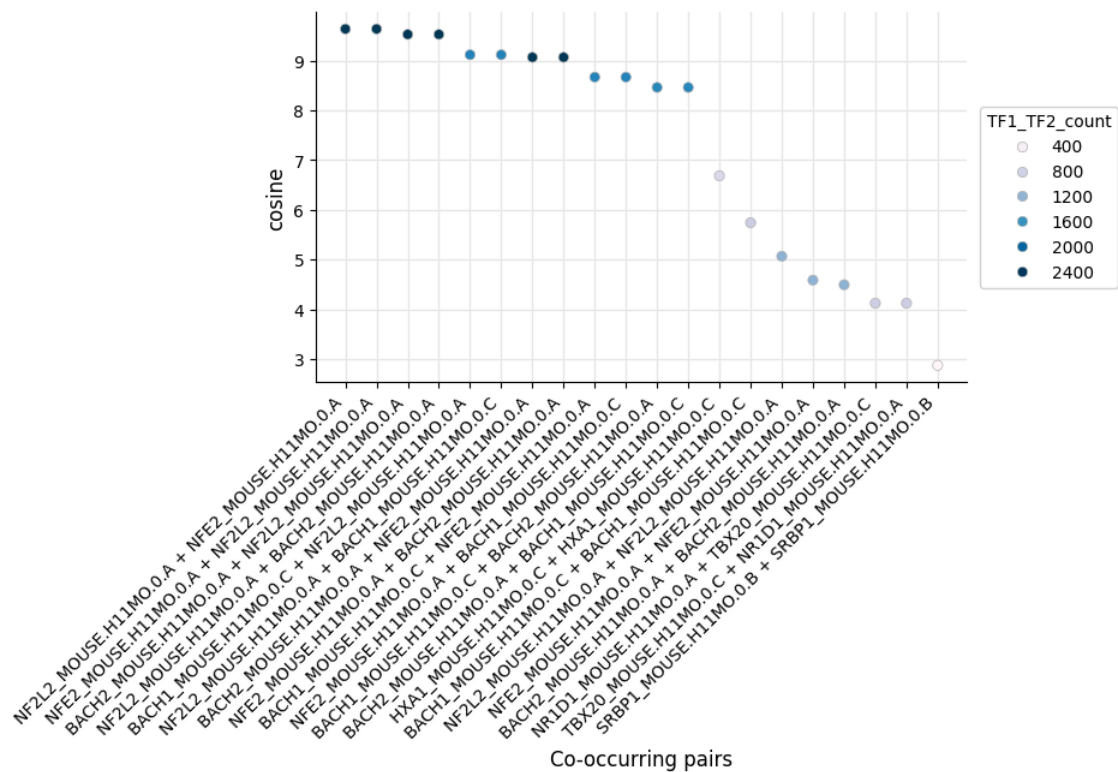
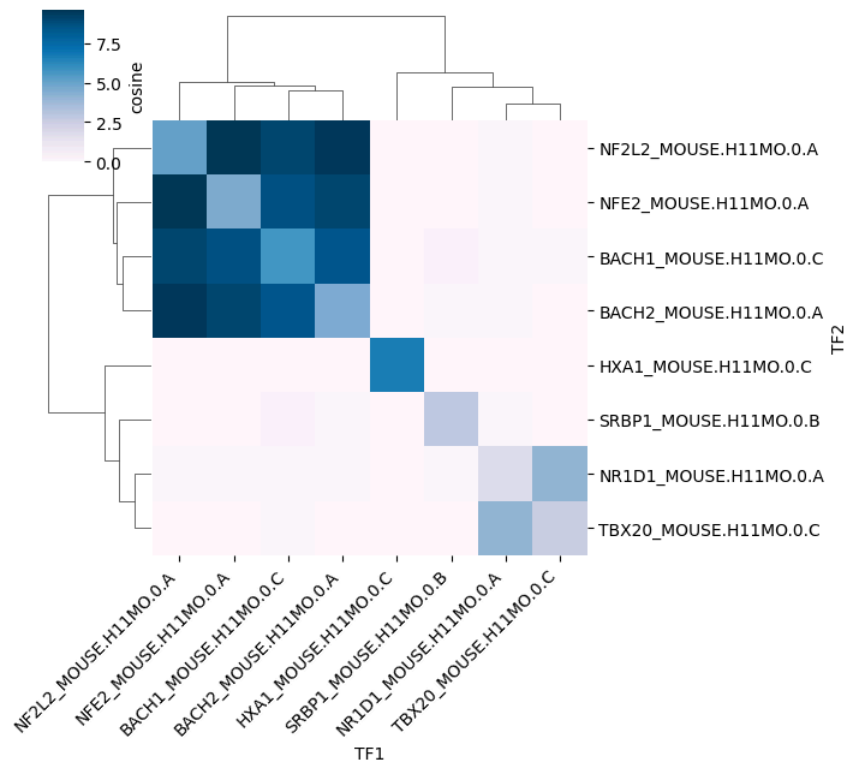
INFO: Scanning for TFBS with 4 thread(s)...
INFO: Progress: 9%
INFO: Progress: 18%
INFO: Progress: 27%
INFO: Progress: 36%
INFO: Progress: 45%
INFO: Progress: 55%
INFO: Progress: 64%
INFO: Progress: 73%
INFO: Progress: 82%
INFO: Progress: 91%
INFO: Finished!
INFO: Processing scanned TFBS
INFO: Identified 49063 TFBS (356 unique names) within given regions
INFO: Setting up binding sites for counting
INFO: Counting co-occurrences within sites
INFO: Counting co-occurrence within background
INFO: Progress: 12%
INFO: Progress: 20%
INFO: Progress: 32%
INFO: Progress: 40%
INFO: Progress: 52%
INFO: Progress: 64%
INFO: Progress: 72%
INFO: Progress: 88%
INFO: Finished!
INFO: Done finding co-occurrences! Run .market_basket() to estimate significant pairs
INFO: Market basket analysis is done! Results are found in <CombObj>.rules

```

Out[8]:

	TF1	TF2	TF1_TF2_count	TF1_count	TF2_count	cosine	zscore
NF2L2_MOUSE.H11MO.0.A- NFE2_MOUSE.H11MO.0.A	NF2L2_MOUSE.H11MO.0.A	NFE2_MOUSE.H11MO.0.A	2412	238	263	9.640753	449.150368
NFE2_MOUSE.H11MO.0.A- NF2L2_MOUSE.H11MO.0.A	NFE2_MOUSE.H11MO.0.A	NF2L2_MOUSE.H11MO.0.A	2412	263	238	9.640753	449.150368
BACH2_MOUSE.H11MO.0.A- NF2L2_MOUSE.H11MO.0.A	BACH2_MOUSE.H11MO.0.A	NF2L2_MOUSE.H11MO.0.A	2407	268	238	9.530599	382.932831
NF2L2_MOUSE.H11MO.0.A- BACH2_MOUSE.H11MO.0.A	NF2L2_MOUSE.H11MO.0.A	BACH2_MOUSE.H11MO.0.A	2407	238	268	9.530599	382.932831
BACH1_MOUSE.H11MO.0.C- NF2L2_MOUSE.H11MO.0.A	BACH1_MOUSE.H11MO.0.C	NF2L2_MOUSE.H11MO.0.A	1706	147	238	9.120778	378.156852
...	...	...	...	...	...	...	...
EHF_MOUSE.H11MO.0.B- HXA1_MOUSE.H11MO.0.C	EHF_MOUSE.H11MO.0.B	HXA1_MOUSE.H11MO.0.C	1	268	105	0.005961	-3.512323
HXA1_MOUSE.H11MO.0.C- BACH2_MOUSE.H11MO.0.A	HXA1_MOUSE.H11MO.0.C	BACH2_MOUSE.H11MO.0.A	1	105	268	0.005961	-3.851957
HXA1_MOUSE.H11MO.0.C- EHF_MOUSE.H11MO.0.B	HXA1_MOUSE.H11MO.0.C	EHF_MOUSE.H11MO.0.B	1	105	268	0.005961	-3.512323
ELF3_MOUSE.H11MO.0.B- HXA1_MOUSE.H11MO.0.C	ELF3_MOUSE.H11MO.0.B	HXA1_MOUSE.H11MO.0.C	1	311	105	0.005534	-3.673924
HXA1_MOUSE.H11MO.0.C- ELF3_MOUSE.H11MO.0.B	HXA1_MOUSE.H11MO.0.C	ELF3_MOUSE.H11MO.0.B	1	105	311	0.005534	-3.673924

105225 rows x 7 columns



```
In [9]: # look at all TFBS sites containing STAT3
# this way returns a list of TFBS object
stat3_tfbs1 = [x for x in opened.TFBS if "STAT3" in x.name]
stat3_tfbs1[:5]

len(stat3_tfbs1)
## 47 stat3 sites

# get all STAT3 motif variants
stat3_variants1 = sorted(set(x.name for x in opened.TFBS if "STAT3" in x.name))
```

```
print(stat3_variants1)

# filter the significant co-occurrence pairs to those with STAT3 co-occurrence
sorted(set(x.name for x in opened.TFBS if "STAT3" in x.name))

stat3_pairs1 = opened.select_TF_rules(["STAT3_MOUSE.H11M0.0.A"])

stat3_pairs1.rules.head()
```

```
Out[9]: [chr1 36569948 36569960 STAT3_MOUSE.H11M0.0.A 9.3781 +,
chr1 36571689 36571701 STAT3_MOUSE.H11M0.0.A 9.33884 -,
chr1 36572133 36572145 STAT3_MOUSE.H11M0.0.A 9.37977 +,
chr1 59316695 59316707 STAT3_MOUSE.H11M0.0.A 9.3636 +,
chr1 62724672 62724684 STAT3_MOUSE.H11M0.0.A 9.10253 +]
```

```
Out[9]: 200
```

```
['STAT3_MOUSE.H11M0.0.A']
```

```
Out[9]: ['STAT3_MOUSE.H11M0.0.A']
```

```
INFO: Selected 1 rules
```

```
INFO: Creating subset of object
```

```
Out[9]:
```

	TF1	TF2	TF1_TF2_count	TF1_count	TF2_count	cosine	zscore
	STAT3_MOUSE.H11M0.0.A- STAT3_MOUSE.H11M0.0.A	STAT3_MOUSE.H11M0.0.A	STAT3_MOUSE.H11M0.0.A	209	200	200	1.045 70.044287

```
In [24]: # do differential analysis between background and differentially expressed peaks
```

```
# combine the 2 objects, positive log2fc will show TF pairs enriched in the opened regions (first object)
opened.set_prefix("opened")
background.set_prefix("background")

diff = DiffCombObj([opened, background],
                    join = 'inner' # only TFs present on both objects are kept
                    )

# normalise
diff.normalize()

# calculate fold changes
diff.calculate_foldchanges() # use default pseudo = 0.01, defaults to cosine

print(diff.rules.columns)

# print table
print("All comparisons:")
pd.set_option('display.max_columns', None) # show all columns
pd.set_option('display.width', None) # auto-detect width
pd.set_option('display.max_colwidth', None) # don't truncate column content
diff.rules.sort_values('opened/background_cosine_log2fc', ascending=False)

print("Enriched in opened regions:")
diff.rules[diff.rules['opened/background_cosine_log2fc'] > 0
           ].sort_values('opened/background_cosine_log2fc', ascending = False
                        ).head(15)

print("STAT3 is the 13th most enriched TF pair out of all TF pairs")

# plot
diff.plot_heatmap()
```

```
INFO: Calculating foldchange for contrast: opened / background
INFO: The calculated log2fc's are found in the rules table (<DiffCombObj>.rules)
Index(['TF1', 'TF2', 'opened_cosine', 'background_cosine',
       'opened/background_cosine_log2fc'],
      dtype='object')
All comparisons:
```


Out[24]:

		TF1	TF2	opened_cosine	background_cosine	opened/background_cosine_log2fc
INSM1_MOUSE.H11MO.0.C-ZEB1_MOUSE.H11MO.0.B	INSM1_MOUSE.H11MO.0.C	ZEB1_MOUSE.H11MO.0.B		1.272882	0.088297	3.706093
ZEB1_MOUSE.H11MO.0.B-INSM1_MOUSE.H11MO.0.C	ZEB1_MOUSE.H11MO.0.B	INSM1_MOUSE.H11MO.0.C		1.272882	0.088297	3.706093
SRBP1_MOUSE.H11MO.0.B-SRBP1_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B		2.501497	0.217027	3.467611
OVOL1_MOUSE.H11MO.0.C-BRAC_MOUSE.H11MO.0.B	OVOL1_MOUSE.H11MO.0.C	BRAC_MOUSE.H11MO.0.B		1.058130	0.089813	3.419721
BRAC_MOUSE.H11MO.0.B-OVOL1_MOUSE.H11MO.0.C	BRAC_MOUSE.H11MO.0.B	OVOL1_MOUSE.H11MO.0.C		1.058130	0.089813	3.419721
...	...	...	...	...	...	...
HNF6_MOUSE.H11MO.0.A-HXB7_MOUSE.H11MO.0.C	HNF6_MOUSE.H11MO.0.A	HXB7_MOUSE.H11MO.0.C		0.026425	5.902386	-7.342660
DDIT3_MOUSE.H11MO.0.C-CEBPG_MOUSE.H11MO.0.B	DDIT3_MOUSE.H11MO.0.C	CEBPG_MOUSE.H11MO.0.B		0.021506	7.209688	-7.840181
CEBPG_MOUSE.H11MO.0.B-DDIT3_MOUSE.H11MO.0.C	CEBPG_MOUSE.H11MO.0.B	DDIT3_MOUSE.H11MO.0.C		0.021506	7.209688	-7.840181
DDIT3_MOUSE.H11MO.0.C-ATF4_MOUSE.H11MO.0.A	DDIT3_MOUSE.H11MO.0.C	ATF4_MOUSE.H11MO.0.A		0.020952	7.289634	-7.881637
ATF4_MOUSE.H11MO.0.A-DDIT3_MOUSE.H11MO.0.C	ATF4_MOUSE.H11MO.0.A	DDIT3_MOUSE.H11MO.0.C		0.020952	7.289634	-7.881637

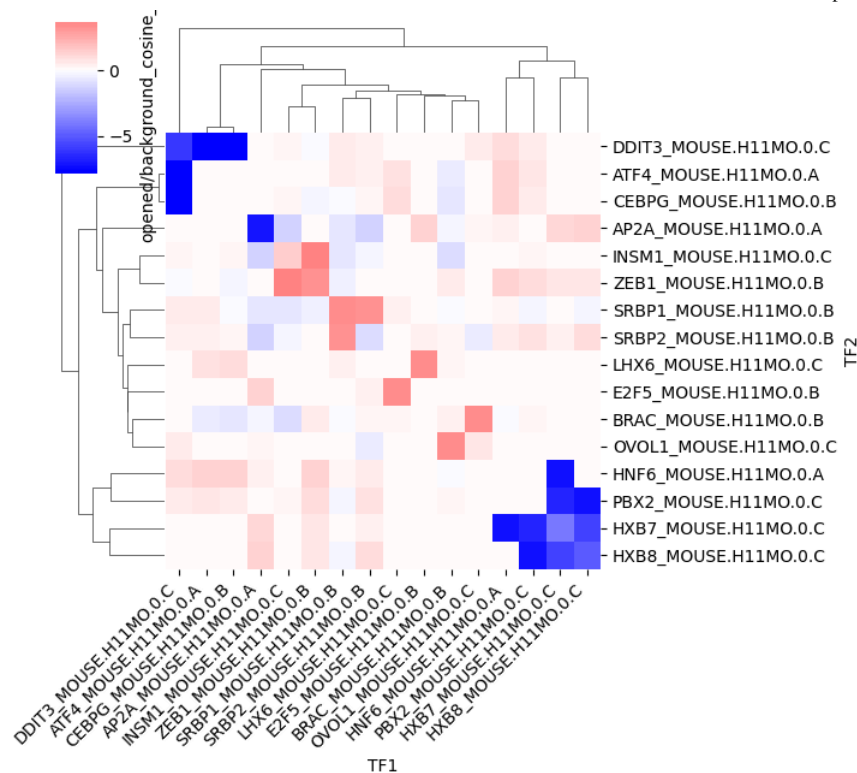
105225 rows x 5 columns

Enriched in opened regions:

Out[24]:

		TF1	TF2	opened_cosine	background_cosine	opened/background_cosine_log2fc
	ZEB1_MOUSE.H11MO.0.B-INSM1_MOUSE.H11MO.0.C	ZEB1_MOUSE.H11MO.0.B	INSM1_MOUSE.H11MO.0.C	1.272882	0.088297	3.706093
	INSM1_MOUSE.H11MO.0.C-ZEB1_MOUSE.H11MO.0.B	INSM1_MOUSE.H11MO.0.C	ZEB1_MOUSE.H11MO.0.B	1.272882	0.088297	3.706093
	SRBP1_MOUSE.H11MO.0.B-SRBP1_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B	2.501497	0.217027	3.467611
	OVOL1_MOUSE.H11MO.0.C-BRAC_MOUSE.H11MO.0.B	OVOL1_MOUSE.H11MO.0.C	BRAC_MOUSE.H11MO.0.B	1.058130	0.089813	3.419721
	BRAC_MOUSE.H11MO.0.B-OVOL1_MOUSE.H11MO.0.C	BRAC_MOUSE.H11MO.0.B	OVOL1_MOUSE.H11MO.0.C	1.058130	0.089813	3.419721
	E2F5_MOUSE.H11MO.0.B-LHX6_MOUSE.H11MO.0.C	E2F5_MOUSE.H11MO.0.B	LHX6_MOUSE.H11MO.0.C	0.160953	0.006162	3.402942
	LHX6_MOUSE.H11MO.0.C-E2F5_MOUSE.H11MO.0.B	LHX6_MOUSE.H11MO.0.C	E2F5_MOUSE.H11MO.0.B	0.160953	0.006162	3.402942
	SRBP2_MOUSE.H11MO.0.B-SRBP1_MOUSE.H11MO.0.B	SRBP2_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B	1.391726	0.133067	3.292436
	SRBP1_MOUSE.H11MO.0.B-SRBP2_MOUSE.H11MO.0.B	SRBP1_MOUSE.H11MO.0.B	SRBP2_MOUSE.H11MO.0.B	1.391726	0.133067	3.292436
	ZEB1_MOUSE.H11MO.0.B-ZEB1_MOUSE.H11MO.0.B	ZEB1_MOUSE.H11MO.0.B	ZEB1_MOUSE.H11MO.0.B	1.835002	0.181615	3.267337
	TBX20_MOUSE.H11MO.0.C-NR1D1_MOUSE.H11MO.0.A	TBX20_MOUSE.H11MO.0.C	NR1D1_MOUSE.H11MO.0.A	3.242339	0.353013	3.163384
	NR1D1_MOUSE.H11MO.0.A-TBX20_MOUSE.H11MO.0.C	NR1D1_MOUSE.H11MO.0.A	TBX20_MOUSE.H11MO.0.C	3.242339	0.353013	3.163384
	STAT3_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	1.335599	0.140501	3.160406
	HINFP_MOUSE.H11MO.0.C-PITX1_MOUSE.H11MO.0.C	HINFP_MOUSE.H11MO.0.C	PITX1_MOUSE.H11MO.0.C	0.152936	0.008568	3.133381
	PITX1_MOUSE.H11MO.0.C-HINFP_MOUSE.H11MO.0.C	PITX1_MOUSE.H11MO.0.C	HINFP_MOUSE.H11MO.0.C	0.152936	0.008568	3.133381

STAT3 is the 13th most enriched TF pair out of all TF pairs



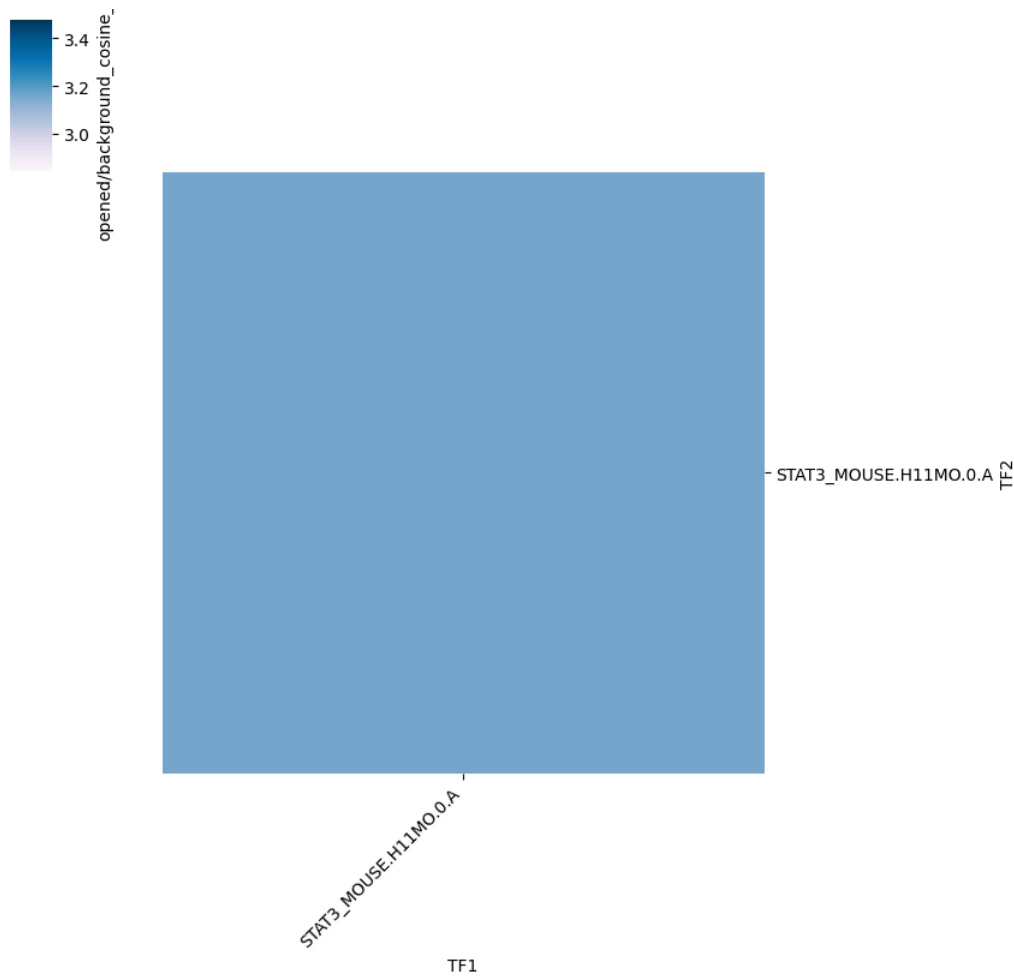
```
In [11]: # filter for stat3 pairs enriched in opened regions
diff_stat3 = diff.select_TF_rules(["STAT3_MOUSE.H11MO.0.A"])
print(diff_stat3.rules.columns)

# inspect positive log2fc (enriched in opened vs all)
# print table
print("Enriched in opened regions:")
pd.set_option('display.max_columns', None) # show all columns
pd.set_option('display.width', None) # auto-detect width
pd.set_option('display.max_colwidth', None) # don't truncate column content
diff_stat3.rules.sort_values('opened/background_cosine_log2fc', ascending=False)

# plot
diff_stat3.plot_heatmap()
```

```
INFO: Selected 1 rules
INFO: Creating subset of object
Index(['TF1', 'TF2', 'opened_cosine', 'background_cosine',
      'opened/background_cosine_log2fc'],
      dtype='object')
Enriched in opened regions:
```

	TF1	TF2	opened_cosine	background_cosine	opened/background_cosine_log2fc
STAT3_MOUSE.H11MO.0.A- STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	1.335599	0.140501	3.160406



```
In [18]: # realised that if filter in a different way, get non-sf tf pairs as well
# select rules only keeps significant ones - check this is why others aren't showing
diff_stat3_all = diff.rules[
    diff.rules.index.str.contains('STAT3')
].sort_values('opened/background_cosine_log2fc', ascending=False)

print(f"Total STAT3 rows: {len(diff_stat3_all)}")

# Remove duplicates (keep only one direction per pair)
diff_stat3_unique = diff_stat3_all[diff_stat3_all['TF1'] <= diff_stat3_all['TF2']]
print(f"Unique STAT3 pairs: {len(diff_stat3_unique)}")

# Look at top pairs
print("\nTop 10 enriched STAT3 pairs:")
diff_stat3_unique.head(10)

# Check how many are enriched vs depleted
enriched = diff_stat3_unique[diff_stat3_unique['opened/background_cosine_log2fc'] > 0]
depleted = diff_stat3_unique[diff_stat3_unique['opened/background_cosine_log2fc'] < 0]
print(f"\nEnriched in opened: {len(enriched)}")
print(f"Depleted in opened: {len(depleted)}")

# save csv
enriched.to_csv(os.path.join(dir_path, "TF-COMBS/STAT3_TF_pairs_enrichedInDiffAccessOCRs_500bpwindow.csv"))
```

Total STAT3 rows: 677

Unique STAT3 pairs: 339

Top 10 enriched STAT3 pairs:

Out[18]:

		TF1	TF2	opened_cosine	background_cosine	opened/background_cosine_log2fc
STAT3_MOUSE.H11MO.0.A- STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A		1.335599	0.140501	3.160406
OVOL2_MOUSE.H11MO.0.C- STAT3_MOUSE.H11MO.0.A	OVOL2_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A		0.115671	0.027294	1.752654
E2F2_MOUSE.H11MO.0.B- STAT3_MOUSE.H11MO.0.A	E2F2_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A		0.053152	0.015877	1.287154
STAT3_MOUSE.H11MO.0.A- ZKSC1_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	ZKSC1_MOUSE.H11MO.0.A		0.090729	0.032199	1.255191
P73_MOUSE.H11MO.0.B- STAT3_MOUSE.H11MO.0.A	P73_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A		0.074517	0.027396	1.176372
OVOL1_MOUSE.H11MO.0.C- STAT3_MOUSE.H11MO.0.A	OVOL1_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A		0.043237	0.013783	1.162527
HXA13_MOUSE.H11MO.0.C- STAT3_MOUSE.H11MO.0.A	HXA13_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A		0.070287	0.027616	1.093829
P63_MOUSE.H11MO.0.B- STAT3_MOUSE.H11MO.0.A	P63_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A		0.087988	0.036099	1.087854
STAT3_MOUSE.H11MO.0.A- XBP1_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	XBP1_MOUSE.H11MO.0.C		0.028557	0.008522	1.057716
NKX61_MOUSE.H11MO.0.A- STAT3_MOUSE.H11MO.0.A	NKX61_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A		0.091270	0.038854	1.051652

Enriched in opened: 228

Depleted in opened: 111

```
In [19]: # look at structure of the objects
print("Opened object rules shape:", opened.rules.shape)
print("\nColumns in opened.rules:")
print(opened.rules.columns)

print("Diff object rules shape:", diff.rules.shape)
print("\nColumns in diff.rules:")
print(diff.rules.columns)
```

Opened object rules shape: (105225, 7)

Diff object rules shape: (105225, 5)

Columns in opened.rules:

```
Index(['TF1', 'TF2', 'TF1_TF2_count', 'TF1_count', 'TF2_count', 'cosine',
      'zscore'],
      dtype='object')
```

Columns in diff.rules:

```
Index(['TF1', 'TF2', 'opened_cosine', 'background_cosine',
      'opened/background_cosine_log2fc'],
      dtype='object')
```

```
In [25]: # generate table for enriched stat3 pairs that has the association rule metrics in it

# get metrics from both objects (excluding cosine since it's already in diff.rules)
opened_metrics = opened.rules[['TF1_TF2_count', 'TF1_count', 'TF2_count', 'zscore']]
opened_metrics.columns = ['opened_cooccur_count', 'opened_TF1_count', 'opened_TF2_count', 'opened_zscore']

background_metrics = background.rules[['TF1_TF2_count', 'TF1_count', 'TF2_count', 'zscore']]
background_metrics.columns = ['background_cooccur_count', 'background_TF1_count', 'background_TF2_count', 'background_zscore']

# merge everything
combined = diff.rules.copy()
combined = combined.join(opened_metrics, how='left')
combined = combined.join(background_metrics, how='left')

# calculate key significance metrics
n_opened_regions = 1385
n_background_regions = 27041

combined['opened_pct_regions'] = (combined['opened_cooccur_count'] / n_opened_regions) * 100
combined['background_pct_regions'] = (combined['background_cooccur_count'] / n_background_regions) * 100

# enrichment ratio (raw fold change, not log2)
combined['enrichment_ratio'] = combined['opened_cosine'] / combined['background_cosine']

# difference in z-scores (higher = more statistically different)
combined['zscore_diff'] = combined['opened_zscore'] - combined['background_zscore']

# absolute counts for context
combined['count_difference'] = combined['opened_cooccur_count'] - combined['background_cooccur_count']

# final table with most relevant columns
significance_table = combined[['TF1', 'TF2',
                                # opened metrics
```

```

'opened_cooccur_count', 'opened_pct_regions', 'opened_cosine', 'opened_zscore',
# background metrics
'background_cooccur_count', 'background_pct_regions', 'background_cosine', 'background_zscore',
# combined comparison metrics
'opened/background_cosine_log2fc', # magnitude of enrichment
'enrichment_ratio', # fold change (non-log)
'zscore_diff', # statistical difference
'count_difference' # absolute difference in co-occurrences
]]

# filter for STAT3 enriched pairs and sort
stat3_enriched = significance_table[
    (significance_table.index.str.contains('STAT3')) &
    (significance_table['opened/background_cosine_log2fc'] > 0)
].sort_values('opened/background_cosine_log2fc', ascending=False)

# remove duplicates
stat3_enriched_unique = stat3_enriched[stat3_enriched['TF1'] <= stat3_enriched['TF2']]

print(f"STAT3 enriched pairs: {len(stat3_enriched_unique)}")
stat3_enriched_unique.head(20)

# save
#enriched.to_csv(os.path.join(dir_path, "TF-COMBS/STAT3_TF_pairs_enrichedInDiffAccess0CRs_500bpwindow_allmetrics.csv"))

```

STAT3 enriched pairs: 228

Out[25]:		TF1	TF2	opened_cooccur_count	opened_pct_regions	opened_cosine	opened_zscore	background_zscore
	STAT3_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	209	15.090253	1.335599	70.044287	
	OVOL2_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	OVOL2_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	4	0.288809	0.115671	3.880570	
	E2F2_MOUSE.H11MO.0.B-STAT3_MOUSE.H11MO.0.A	E2F2_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A	2	0.144404	0.053152	2.577193	
	STAT3_MOUSE.H11MO.0.A-ZKSC1_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	ZKSC1_MOUSE.H11MO.0.A	13	0.938628	0.090729	1.073908	
	P73_MOUSE.H11MO.0.B-STAT3_MOUSE.H11MO.0.A	P73_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A	9	0.649819	0.074517	1.017094	
	OVOL1_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	OVOL1_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	4	0.288809	0.043237	0.214535	
	HXA13_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	HXA13_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	7	0.505415	0.070287	1.423938	
	P63_MOUSE.H11MO.0.B-STAT3_MOUSE.H11MO.0.A	P63_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A	9	0.649819	0.087988	2.518959	
	STAT3_MOUSE.H11MO.0.A-XBP1_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	XBP1_MOUSE.H11MO.0.C	1	0.072202	0.028557	0.662266	
	NKX61_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	NKX61_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	13	0.938628	0.091270	1.454030	
	SRF_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	SRF_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	7	0.505415	0.060427	0.758621	
	GF1_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	GF1_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	9	0.649819	0.077290	1.086611	
	CLOCK_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	CLOCK_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	10	0.722022	0.072020	0.720116	
	JUN_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	JUN_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	22	1.588448	0.108606	0.470004	
	SPI1_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	SPI1_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	50	3.610108	0.169979	-0.032191	
	ANDR_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	ANDR_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	13	0.938628	0.090729	1.250689	
	SPIB_MOUSE.H11MO.0.A-STAT3_MOUSE.H11MO.0.A	SPIB_MOUSE.H11MO.0.A	STAT3_MOUSE.H11MO.0.A	47	3.393502	0.162184	0.345101	
	DDIT3_MOUSE.H11MO.0.C-STAT3_MOUSE.H11MO.0.A	DDIT3_MOUSE.H11MO.0.C	STAT3_MOUSE.H11MO.0.A	5	0.361011	0.046726	0.271503	
	STAT3_MOUSE.H11MO.0.A-TBX3_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A	TBX3_MOUSE.H11MO.0.B	3	0.216606	0.046273	0.635999	
	BRAC_MOUSE.H11MO.0.B-STAT3_MOUSE.H11MO.0.A	BRAC_MOUSE.H11MO.0.B	STAT3_MOUSE.H11MO.0.A	16	1.155235	0.081746	-0.375948	

```

In [29]: # get the metrics used by market basket to determine what is an association or not (only the ones that pass the thresholding are kept)
# want:
# motif pvalue = statistical significance/motif match
# support = frequency of tf pair. count(A&B)/total number of windows i.e. baskets
# confidence P(TF2|TF1). count(A&B)/count(A)
# lift = enrichment over random. support(A&B)/ [support(A)*support(B)]

```

```

# inside the opened or background object there is:
# rules = filtered association table with TF1, TF2, support, confidence, lift, count
# pair_counts = square matrix with TF1, TF2, values = number of regions where they co-occur
# TFBS = list of individual TFBSs

# extract raw co-occurrence counts
STAT3 = "STAT3_MOUSE.H11M0.0.A"

stat3_raw = (opened.pair_counts
              .loc[STAT3]
              .reset_index()
              .rename(columns={"index": "TF2", STAT3: "TF1_TF2_count"}))
stat3_raw.head()

#pair_counts = opened.counts["TF_counts"]

print("\nColumns in pair_counts.rules:")
print(diff.pair_counts.columns)

```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[29], line 17
      1 # get the metrics used by market basket to determine what is an association or not (only the ones that pass the thresholding are kept)
      2 # want:
      3     # motif pvalue = statistical significance/motif match
      4     (...)     12
      5 # extract raw co-occurrence counts
      6 STAT3 = "STAT3_MOUSE.H11M0.0.A"
----> 16 stat3_raw = (opened.pair_counts
      17                .loc[STAT3]
      18                .reset_index()
      19                .rename(columns={"index": "TF2", STAT3: "TF1_TF2_count"}))
      20            )
      21 stat3_raw.head()
      22 #pair_counts = opened.counts["TF_counts"]

AttributeError: 'numpy.ndarray' object has no attribute 'loc'

```

```

In [33]: import pandas as pd
import numpy as np

STAT3 = "STAT3_MOUSE.H11M0.0.A"

# core data
tf_names = opened.count_names
pair_counts = opened.pair_counts      # numpy array
tf_counts = opened.TF_counts          # numpy array
N = max(tf_counts)                    # total windows

i = tf_names.index(STAT3)

rows = []
for j, tf2 in enumerate(tf_names):
    c_xy = pair_counts[i, j]
    c_x = tf_counts[i]
    c_y = tf_counts[j]

    support = c_xy / N if N > 0 else 0
    confidence = c_xy / c_x if c_x > 0 else 0
    lift = (c_xy / N) / ((c_x / N) * (c_y / N)) if c_x > 0 and c_y > 0 else 0

    rows.append((STAT3, tf2, c_xy, support, confidence, lift))

stat3_metrics = pd.DataFrame(
    rows,
    columns=["TF1", "TF2", "cooccurrence_count", "support", "confidence", "lift"]
)

# remove STAT3-STAT3
stat3_metrics = stat3_metrics[stat3_metrics["TF2"] != STAT3]

# sort by lift (or confidence)
stat3_metrics.sort_values(
    ["lift", "confidence", "support"],
    ascending=[False, False, False]
).head(20)

# STAT3-STAT3 homotypic interactions were probably excluded by default from this

```

Out[33]:

	TF1	TF2	cooccurrence_count	support	confidence	lift
230	STAT3_MOUSE.H11MO.0.A	OVOL2_MOUSE.H11MO.0.C	4	0.003182	0.020	4.190000
51	STAT3_MOUSE.H11MO.0.A	E2F2_MOUSE.H11MO.0.B	2	0.001591	0.010	2.095000
232	STAT3_MOUSE.H11MO.0.A	P63_MOUSE.H11MO.0.B	9	0.007160	0.045	1.178438
52	STAT3_MOUSE.H11MO.0.A	E2F3_MOUSE.H11MO.0.A	8	0.006364	0.040	1.169302
340	STAT3_MOUSE.H11MO.0.A	XBP1_MOUSE.H11MO.0.C	1	0.000796	0.005	1.047500
318	STAT3_MOUSE.H11MO.0.A	TBX3_MOUSE.H11MO.0.B	3	0.002387	0.015	1.047500
127	STAT3_MOUSE.H11MO.0.A	HXA13_MOUSE.H11MO.0.C	7	0.005569	0.035	1.023140
55	STAT3_MOUSE.H11MO.0.A	E2F6_MOUSE.H11MO.0.A	11	0.008751	0.055	1.016691
322	STAT3_MOUSE.H11MO.0.A	TEAD4_MOUSE.H11MO.0.A	9	0.007160	0.045	0.992368
203	STAT3_MOUSE.H11MO.0.A	NFYB_MOUSE.H11MO.0.A	6	0.004773	0.030	0.992368
53	STAT3_MOUSE.H11MO.0.A	E2F4_MOUSE.H11MO.0.A	17	0.013524	0.085	0.971318
109	STAT3_MOUSE.H11MO.0.A	GF1_MOUSE.H11MO.0.C	9	0.007160	0.045	0.942750
50	STAT3_MOUSE.H11MO.0.A	E2F1_MOUSE.H11MO.0.A	14	0.011138	0.070	0.916563
233	STAT3_MOUSE.H11MO.0.A	P73_MOUSE.H11MO.0.B	9	0.007160	0.045	0.883828
212	STAT3_MOUSE.H11MO.0.A	NKX61_MOUSE.H11MO.0.A	13	0.010342	0.065	0.869202
27	STAT3_MOUSE.H11MO.0.A	CDX2_MOUSE.H11MO.0.A	8	0.006364	0.040	0.866897
3	STAT3_MOUSE.H11MO.0.A	ANDR_MOUSE.H11MO.0.A	13	0.010342	0.065	0.860053
350	STAT3_MOUSE.H11MO.0.A	ZKSC1_MOUSE.H11MO.0.A	13	0.010342	0.065	0.860053
315	STAT3_MOUSE.H11MO.0.A	TBP_MOUSE.H11MO.0.A	5	0.003978	0.025	0.849324
211	STAT3_MOUSE.H11MO.0.A	NKX32_MOUSE.H11MO.0.B	4	0.003182	0.020	0.785625

In []: